11. Hata Mesajlarını ve Stack Trace'i AI ile Okuma ve Akıl Yürütme Sanatı

Terminaldeki Kırmızı Yazılardan Korkmayı Bırakıp Dedektif Gibi İz Sürmek
Bölüm 2 • Zorluk Derecesi: Temel / Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Junior mühendisler terminaldeki 50 satırlık kırmızı hata mesajını görünce neden panikler ve AI bu panikten nasıl kurtarır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Stack Trace düşmanınız değil; hatanın nerede, ne zaman ve hangi çağrı zincirinde gerçekleştiğini söyleyen yol haritasıdır. - 📌 AI'a sadece 'Kod çalışmıyor' demek yerine tüm stack trace'i ve ilgili fonksiyon kodunu vermek çözümü 10 saniyeye indirir. - 📌 En üstteki kütüphane satırlarına değil, kendi yazdığınız kodun bulunduğu satır numarasına odaklanın.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir uçağın kokpitini hayal edin. Aniden kırmızı bir ikaz ışığı yanıp sönüyor ve siren çalıyor.

Acemi bir pilot ne yapar? Panikleyip ellerini havaya kaldırır veya gözünü kapatır. Tecrübeli bir kaptan pilot ne yapar? Sakinlikle ikaz panelindeki hata kodunu okur: *"Motor 2 yağ basıncı uyarısı, Sayfa 4B kontrol listesini açın."*

Terminalinizde patlayan o 50 satırlık kırmızı Stack Trace de uçağın ikaz panelidir. O kırmızı yazılar size kızmıyor; sadece: *"Filanca dosyanın 42. satırında tanımsız bir objenin özelliğini okumaya çalıştın"* diyor.

Yapay zekayı yanınıza yardımcı pilot olarak aldığınızda, o karmaşık logları 5 saniyede tercüme edebilirsiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Stack Trace Okuma Filtresi:

```
[50 Satırlık Kırmızı Terminal Çıktısı]
              |
              ▼
[1. Hata Türü & Mesajı]: TypeError: Cannot read properties of undefined (reading
'email')
              |
              ▼
[2. Kütüphane Gürültüsünü Ele]: /node_modules/express/lib/router/... (ATLA!)
              |
              ▼
[3. Kendi Kodunun İlk Satırı]: at UserService.getUserProfile (src/services/
user.ts:42:18)
              |
              ▼
[4. AI'a Beslenecek Odak]: "src/services/user.ts dosyasının 42. satırındaki çağrı
bağlamı"
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Debugging ve Kök Neden Analiz Uzmanı
Uygulamamız aşağıdaki hatayı fırlattı.

## HATA MESAJI & STACK TRACE:
```text
TypeError: Cannot read properties of undefined (reading 'name')
    at formatCustomerCard (/app/src/utils/card.ts:34:25)
    at Array.map (<anonymous>)
    at getCustomersDashboard (/app/src/controllers/dashboard.ts:18:32)
````

İLGİLİ KOD (src/utils/card.ts - Satır 30-40):

```
export function formatCustomerCard(customer: Customer) {
  return {
    id: customer.id,
    displayName: customer.profile.name.toUpperCase(),
  };
}
```
```

GÖREV:

1. Bu hatanın gerçek kök nedeni (Root Cause) nedir?
2. `profile` veya `name` alanının eksik geldiği durumları güvenle ele alacak defansif düzeltmeyi yap.
3. Bu durumun gelecekte tekrar etmemesi için TypeScript tipini güncelle.

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Hata Fırlatan Kod: Profil veya Name tanımsız olduğunda patlar
export function formatCustomerCardNaive(customer: any) {
  // Eğer customer.profile null veya undefined ise uygulama çöker!
  return {
    id: customer.id,
    displayName: customer.profile.name.toUpperCase(),
  };
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```typescript
export interface CustomerProfile {
  name?: string;
  avatarUrl?: string;
}

export interface Customer {
  id: string;
  profile?: CustomerProfile | null;
}

export interface FormattedCustomerCard {
  id: string;
  displayName: string;
}

export function formatCustomerCard(customer: Customer): FormattedCustomerCard {
  // Optional Chaining (?.) ve Nullish Coalescing (??) ile tam koruma
  const rawName = customer.profile?.name?.trim();
  const displayName = rawName ? rawName.toUpperCase() : "İSİMSİZ MÜŞTERİ";

  return {
    id: customer.id,
    displayName
  };
}
```
```

💡 **Kod Analizi:** Stack trace incelendiğinde `profile` nesnesinin null geldiği hemen anlaşılır. `customer.profile?.name` ile güvenli zincirleme yapılmış ve varsayılan bir geri dönüş değeri (`İSİMSİZ MÜŞTERİ`) atanarak çökme engellenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Hata çıktısını okumadan tüm dosyayı silip baştan yazdırmak.
  - **Çözüm:** Hatanın işaret ettiği dosya ve satır numarasını bulun, sadece o bloğu inceleyin.
- **Tuzak:** Terminaldeki hatayı kopyalamadan AI'a 'kod hata verdi' demek.
  - **Çözüm:** Her zaman hem hata türünü hem de ilk 10 satırlık stack trace'i eksiksiz yapıştırın.
- **Tuzak:** Hatanın sadece sonucunu gizlemek (Örn: Boş try/catch ile hatayı yutmak).
  - **Çözüm:** Hataları asla sessizce yutmayın; loglayın ve kök nedeni düzeltin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 11: Stack Trace Okuma Sanatı - Kök Neden Analizi*

- 🖥️ **Slayt 1:** Stack Trace anatomisi: Hata türü, mesajı ve çağrı yığını.
- 🖥️ **Slayt 2:** node_modules gürültüsünü filtreleyip uygulama kodunu bulmak.
- 🖥️ **Slayt 3:** Optional Chaining (?.) ve Nullish Coalescing (??) ile defansif kodlama.
- 🖥️ **Slayt 4:** Canlı vaka: Production çökmesini AI ile 60 saniyede teşhis edip çözmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Kasıtlı olarak `null.toUpperCase()` hatası fırlatan bir API yazın. Terminalde beliren stack trace'i kopyalayın, AI asistanına besleyip kök neden analizini ve çözümünü tek adımda gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugün çalıştığınız projede bilerek bir hata fırlatın (`throw new Error('Test Hatası')`). Oluşan stack trace'i satır satır inceleyerek çağrı zincirini takip edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir stack trace incelenirken ilk bakılması gereken yer neresidir? - A) En alttaki işletim sistemi çekirdek çağrıları - B) Hata mesajının kendisi (Error Signature) ve kendi proje dizinimize işaret eden ilk satır numarası - C) Bilgisayarın IP adresi - D) package.json dosyasının son satırı > **Doğru Yanıt:** B > **Açıklama:** Hatanın türü (örn. TypeError) ve kendi kaynak kodumuzun yer aldığı dosya/satır numarası sorunun kaynağını doğrudan gösterir.

Soru 2: JavaScript/TypeScript'te `customer?.profile?.name ?? 'Anonim'` ifadesinin sağladığı fayda nedir? - A) Kodu 10 kat hızlandırır - B) Eğer customer veya profile tanımsız (undefined/null) ise çökmeden güvenle 'Anonim' değerini döndürür - C) Sadece sayılarda çalışır - D) Belleği tamamen sıfırlar > **Doğru Yanıt:** B > **Açıklama:** Optional Chaining (`?.`) zincirdeki ara eleman null/undefined ise hata fırlatılmasını önler; Nullish Coalescing (`??`) ise varsayılan değer sağlar.